

Akademia Górniczo-Hutnicza
im. Stanisława Staszica w Krakowie

Wydział Fizyki i Informatyki Stosowanej

Interfejsy użytkownika i biblioteki graficzne

Eksplorator fraktali

Plan pracy

Autorzy

Jakub Piela (kapitan)
Marcel Gładysz
Patryk Pilch

Prowadzący: dr inż. Janusz Malinowski
Temat: własny
Rok akademicki: 2026/2027

1 Opis projektu

Eksplorator fraktali to program w C++ do generowania, interaktywnego oglądania i animowania fraktali z trzech rodzin:

- **fraktale zespolone** – zbiór Mandelbrota i zbiory Julii,
- **fraktale IFS** – atraktory układów przekształceń afinicznych, np. trójkąt Sierpińskiego i paproć Barnsleya,
- **L-systemy** – gramatyki przepisywania napisów rysowane grafiką żółwia, np. krzywa Kocha i rośliny.

Użytkownik przybliża i przesuwa widok myszą, zmienia parametry i palety kolorów w panelu sterowania, a bieżący obraz zapisuje do pliku. Program tworzy też **animację płynnego przejścia („morfingu") między fraktalami IFS**: przeliczając współczynniki przekształceń z jednego fraktala na drugi, generuje serię bitmap zapisywanych na dysku.

Każdy piksel obrazu liczy nasz kod na procesorze. Biblioteki służą wyłącznie do otwarcia okna, obsługi zdarzeń, wyświetlenia gotowego bufora pikseli i narysowania panelu sterowania.

Animacja przejścia oraz – w wersji rozszerzonej – plik scenariusza i fraktale trójwymiarowe nawiązują do niewybranego tematu nr 25 („Generowanie i animacja fraktali"). Nasz projekt obejmuje jego zakres dotyczący fraktali IFS i rozszerza go o fraktale zespolone oraz L-systemy.

2 Założenia wstępne

- **Język i środowisko:** C++17, Visual Studio 2026, Windows 11, kompilacja x64. Projekt budujemy przez CMake, więc kompiluje się też na Linuksie. SFML jest linkowany statycznie, więc program działa bez dodatkowych plików DLL.
- **Biblioteki z wykładu:** SFML 3.1 (okno, zdarzenia, wyświetlenie bufora pikseli), Dear ImGui z nakładką ImGui-SFML (panel sterowania) oraz ImGuiFileDialog (okna wyboru plików).
- **Bez OpenGL i DirectX w naszym kodzie:** obliczenia, kolorowanie, rysowanie linii i rzutowanie fraktali 3D wykonuje nasz kod na CPU.
- **Działanie bez plików wejściowych:** wszystkie parametry można ustawić w panelu. Pliki definicji są opcjonalne i służą do zapisu oraz wczytywania gotowych ustawień.
- **Jeden wątek w wersji podstawowej:** obliczenia wykonuje główny wątek programu. Animację generujemy po jednej klatce na obieg pętli głównej, więc między klatkami okno się odświeża i pokazuje podgląd. Obliczenia wielowątkowe i generowanie animacji w tle są poszerzeniem.
- **Okno** jest skalowalne. Po zmianie rozmiaru obraz jest przeliczany w nowej rozdzielczości, a nie rozciągany.
- **Oddanie projektu:** archiwum `numer_projektu.7z` (kompresja Ultra) z katalogami `DOC`, `SRC` i `BIN`; dokumentacja w PDF zgodna ze wzorcem.

3 Wymagania

3.1 Wymagania podstawowe

1. **Interfejs graficzny:** okno z obszarem fraktala, panel sterowania z polami i suwakami do zmiany parametrów, menu oraz okna wyboru plików. Po zmianie rozmiaru okna obraz jest przeliczany w nowej rozdzielczości, a nie rozciągany.
2. **Zbiór Mandelbrota:** przybliżanie kółkiem myszy w punkt pod kursorem, przesuwanie przeciąganiem, powrót do widoku startowego, regulowana maksymalna liczba iteracji.
3. **Zbiory Julii:** parametr c wybierany kliknięciem na zbiorze Mandelbrota albo wpisywany w panelu.
4. **Kolorowanie zbiorów zespolonych:** płynne (bez widocznych pasów), co najmniej trzy palety do wyboru.
5. **Fraktale IFS:** rysowanie grą w chaos; wbudowane trójkąt Sierpińskiego i paproć Barnsleya; edycja współczynników w panelu oraz wczytywanie i zapis definicji w pliku tekstowym. Kolor punktu zależy od przekształcenia, którym go wyznaczono, a fraktal jest automatycznie skalowany tak, aby mieścił się w oknie i nie był za mały.
6. **Animacja przejścia IFS:** płynne przejście między dwoma fraktalami IFS w zadanej przez użytkownika liczbie klatek, podgląd klatek w trakcie generowania, zapis ponumerowanej serii obrazów PNG lub BMP.
7. **L-systemy:** aksjomat, reguły, kąt i liczba iteracji z panelu lub pliku; rysowanie grafiką żółwia ze stosem rozgałęzień; wbudowane krzywa Kocha, smok Heighwaya i roślina; automatyczne dopasowanie rysunku do okna.
8. **Zapis obrazu:** bieżący widok do pliku PNG lub BMP.

3.2 Wymagania poszerzone

- **Obliczenia wielowątkowe:** zbiory zespolone liczone na wielu wątkach, aby przybliżanie i przesuwanie były płynne; animacja generowana w tle z paskiem postępu i możliwością przerwania.
- **Plik scenariusza:** animacja przez więcej niż dwa fraktale (kolejne fraktale i liczba klatek każdego przejścia).
- **Fraktale IFS 3D:** przekształcenia przestrzenne (12 współczynników), własne rzutowanie perspektywiczne i pozycja obserwatora ustawiana w panelu. Punkty dalsze są ciemniejsze i nie przesłaniają bliższych (własny bufor głębi).
- **Animacja przybliżania** zbioru Mandelbrota lub Julii eksportowana jako seria klatek.
- **Edytor palet:** gradient definiowany punktami kontrolnymi, zapis i wczytywanie palet.
- **Kolejne fraktale zespolone:** Burning Ship, Multibrot (z^n), fraktal Newtona dla wielomianu $z^3 - 1$.
- **Zapis w wysokiej rozdzielczości** (większej niż okno) z wygładzaniem krawędzi (supersampling).

4 Analiza projektu

4.1 Specyfikacja danych wejściowych

Rodzaj	Format	Zawartość
Interakcja	mysz, klawiatura, panel	przybliżanie, przesuwanie, wybór c, parametry fraktali, paleta
Definicja IFS	plik `.ifs`	jedno przekształcenie w wierszu: `a b c d e f p`
L-system	plik `.lsys`	aksjomat, reguły w postaci `F=F+F-F`, kąt, liczba iteracji
Scenariusz animacji	plik `.anim`	(poszerzenie) rozmiar klatek, liczba punktów, kolejne fraktale, liczba klatek każdego przejścia

Przekształcenie IFS ma postać $x' = ax + by + e$, $y' = cx + dy + f$, a p to prawdopodobieństwo, z jakim jest losowane w grze w chaos. Jeśli prawdopodobieństwa w pliku nie sumują się do 1, program je normalizuje. We wszystkich plikach znak `#` zaczyna komentarz, a puste wiersze są pomijane. Przykład – trójkąt Sierpińskiego:

```
# a b c d e f p
0.5 0 0 0.5 0    0    0.3333
0.5 0 0 0.5 0.5  0    0.3333
0.5 0 0 0.5 0.25 0.5  0.3334
```

Plik L-systemu:

```
axiom F
rule F=F+F-F-F+F
angle 90
iterations 4
```

Plik scenariusza (poszerzenie) zawiera rozmiar klatek i liczbę punktów na klatkę, a potem kolejne fraktale w formacie `.ifs` poprzedzone liczbą przekształceń, przedzielone liczbą klatek przejścia:

```
# szerokosc wysokosc punkty_na_klatke
800 600 2000000
# liczba fraktali
2
# fraktal 1: liczba przeksztalcen, potem wiersze a b c d e f p
3
0.5 0 0 0.5 0    0    0.3333
0.5 0 0 0.5 0.5  0    0.3333
0.5 0 0 0.5 0.25 0.5  0.3334
# liczba klatek przejscia do fraktala 2
100
# fraktal 2
4
 0.00  0.00  0.00  0.16  0  0     0.01
 0.85  0.04 -0.04  0.85  0  1.6   0.85
 0.20 -0.26  0.23  0.22  0  1.6   0.07
-0.15  0.28  0.26  0.24  0  0.44  0.07
```

Błędne pliki (zła liczba wartości, niepoprawne liczby, brak sekcji) nie zamykają programu. Wyświetlamy komunikat z numerem wiersza, w którym wystąpił błąd.

4.2 Opis oczekiwanych danych wyjściowych

- **Obraz:** bieżący widok zapisany jako PNG lub BMP w rozmiarze okna albo w zadanej rozdzielczości.
- **Animacja:** seria ponumerowanych plików `klatka_0000.png`, `klatka_0001.png`, ... w wybranym katalogu. Kolejno oglądane tworzą płynną animację.
- **Definicje:** bieżące ustawienia IFS lub L-systemu zapisane do pliku w formatach opisanych powyżej.

4.3 Struktury danych

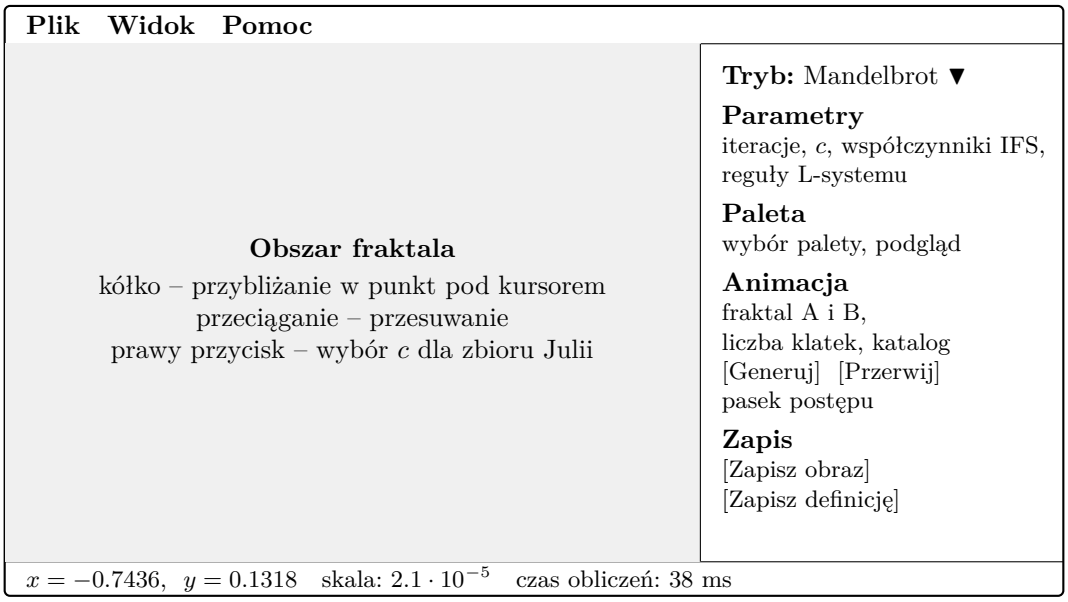
Struktura	Zawartość
`Color`	kolor RGBA, 4 bajty (układ zgodny z `sf::Texture` i `sf::Image`)
`Image`	bufor pikseli `std::vector<Color>` oraz szerokość i wysokość; zapis piksela i odcinka (Bresenham)
`LineReader`	czytnik plików tekstowych wspólny dla parserów: pomija komentarze i puste wiersze, pamięta numer wiersza
`ParseError`	wyjątek z numerem wiersza i komunikatem błędu
`ComplexView`	środek widoku (c_x, c_y), skala (jednostki na piksel), maksymalna liczba iteracji
`Palette`	lista punktów kontrolnych gradientu (pozycja t, kolor)
`AffineMap`	współczynniki a, b, c, d, e, f i prawdopodobieństwo p
`Ifs`	nazwa i `std::vector<AffineMap>`
`AffineMap3D`	macierz 3×3, wektor przesunięcia i p (poszerzenie)
`Morph`	dwa fraktale IFS, liczba klatek, numer bieżącej klatki, katalog i format zapisu
`AnimationScenario`	(poszerzenie) rozmiar klatek, liczba punktów na klatkę, kolejne fraktale, liczby klatek przejść
`LSystem`	aksjomat, reguły `std::map<char, std::string>`, kąt, liczba iteracji
`TurtleState`	pozycja (x, y) i kierunek żółwia; rozgałęzienia na `std::stack<TurtleState>`

Każdy rodzaj fraktala implementuje wspólną klasę bazową `FractalRenderer`. Klasy pochodne to `MandelbrotRenderer`, `JuliaRenderer`, `IfsRenderer` i `LSystemRenderer`. Aplikacja zna tylko ten interfejs, więc zmiana trybu sprowadza się do wyboru innego obiektu:

- `render(Image&)` – liczy i rysuje obraz; nie korzysta z ImGui ani SFML,
- `drawControls()` – rysuje widżety trybu w panelu sterowania,
- `tick()` – wywoływana raz na obieg pętli głównej; służy do pracy rozłożonej na wiele klatek, np. generowania animacji,
- `onZoom`, `onDrag`, `onClick`, `resetView` – nawigacja myszą i powrót do widoku startowego,
- `loadDefinition`, `saveDefinition` – wczytywanie i zapis plików `.ifs` i `.lsys`; błędy zgłaszane przez `ParseError`.

Metody zwracają `true`, gdy obraz trzeba przeliczyć. Aplikacja wywołuje wtedy `render()`.

4.4 Specyfikacja interfejsu użytkownika



Okno składa się z menu (wczytywanie i zapis plików, powrót do widoku startowego, informacje o programie), obszaru fraktala, panelu sterowania po prawej i paska stanu. Pasek stanu pokazuje współrzędne punktu pod kursorem, bieżącą skalę i czas obliczenia klatki. Elementy, których nie da się w danej chwili użyć, są wyszarzone, np. zapis definicji w trybie, który jej nie ma. Pasek postępu animacji pokazuje numer bieżącej klatki, a przycisk *Przerwij* zatrzymuje generowanie po bieżącej klatce. Panel i okna dialogowe tworzymy w Dear ImGui.

4.5 Wyodrębnienie i zdefiniowanie zadań

1. **Rdzeń aplikacji i interfejs:** okno, pętla zdarzeń, panel ImGui, nawigacja myszą, okna dialogowe, zmiana rozmiaru okna, integracja modułów.
2. **Fraktale zespolone:** Mandelbrot i Julia, płynne kolorowanie, palety.
3. **IFS i animacja:** parser plików, gra w chaos, kolorowanie, dopasowanie do okna, przejście między dwoma fraktalami, eksport ponumerowanych klatek.
4. **L-systemy:** parser, przepisywanie napisów, grafika żółwia, rysowanie odcinków w buforze pikseli.
5. **Część wspólna:** klasy `Image`, `LineReader`, `ParseError` i interfejs `FractalRenderer`, zapis PNG i BMP, konfiguracja projektu (CMake, submoduły, formatowanie i analiza kodu), testy.

4.6 Decyzja o wyborze narzędzi programistycznych

- **Visual Studio 2026 i C++17** – znamy je z zajęć. Standard daje `<random>` do losowania w grze w chaos, `<filesystem>` do zapisu klatek oraz `std::thread` i `std::atomic` do poszerzeń wielowątkowych.
- **CMake** – jeden opis budowania dla Visual Studio i Linuksa. Biblioteki są dołączone jako submoduły git w ustalonych wersjach, więc każdy w zespole buduje ten sam kod. Każdy moduł (`core`, `complex`, `ifs`, `lsystem`, `app`) to osobny cel, co ułatwia równoległą pracę i testy bez okna (CTest).
- **clang-format i clang-tidy** – wspólny styl kodu i analiza statyczna. clang-tidy sprawdza tylko nasz kod, nie dołączone biblioteki.

- **SFML 3.1** – poznana na wykładzie i laboratoriach, prosta obsługa okna, zdarzeń i wyświetlania bufora przez `sf::Texture`. `sf::Image` zapisuje obrazy PNG i BMP.
- **Dear ImGui, ImGui-SFML, ImGuiFileDialog** – omawiane na wykładzie. Pozwalają szybko zbudować panel z suwakami, polami liczbowymi i oknami wyboru plików.
- **Git** (wspólne repozytorium) do pracy zespołowej, LaTeX do dokumentacji, 7-Zip do archiwum.

5 Algorytmy

5.1 Zbiory Mandelbrota i Julii

Dla każdego punktu płaszczyzny zespolonej powtarzamy przekształcenie

$$z_{n+1} = z_n^2 + c.$$

W zbiorze Mandelbrota $z_0 = 0$, a c jest współrzędną piksela. W zbiorze Julii c jest stałe dla całego obrazu, a z_0 jest współrzędną piksela. Gdy $|z_n|^2 > 4$, ciąg ucieka do nieskończoności i punkt nie należy do zbioru. Porównujemy kwadrat modułu, więc nie liczymy pierwiastka.

Piksel i płaszczyzna zespolona. Piksel (p_x, p_y) odpowiada punktowi $x = c_x + (p_x - W/2)\,s$, $y = c_y - (p_y - H/2)\,s$, gdzie s to skala. Przy przybliżaniu o czynnik k nowy środek wynosi $\mathbf{m}' = \mathbf{q} + (\mathbf{m} - \mathbf{q})\,k$, gdzie $\mathbf{q}$ to punkt pod kursorem. Dzięki temu punkt pod kursorem zostaje w miejscu.

Płynne kolorowanie. Zamiast całkowitej liczby kroków n używamy wartości ułamkowej

$$\nu = n + 1 - \log_2(\ln |z_n|),$$

którą przeliczamy na kolor z palety. Usuwa to pasy kolorów. Punkty należące do zbioru są czarne.

5.2 Fraktale IFS – gra w chaos

Zaczynamy od dowolnego punktu. W każdym kroku losujemy przekształcenie w_i z prawdopodobieństwem p_i, stosujemy je do punktu i zaznaczamy wynik. Pierwsze 20 punktów pomijamy, bo nie leżą jeszcze na atraktorze.

Prawdopodobieństwa p_i pochodzą z pliku albo z panelu. Przy ręcznej edycji panel podpowiada wartości proporcjonalne do pola, jakie przekształcenie zachowuje:

$$p_i = \frac{\max(|a_i d_i - b_i c_i|,\ \varepsilon)}{\sum_j \max(|a_j d_j - b_j c_j|,\ \varepsilon)}.$$

Przy takich wartościach punkty rozkładają się na fraktalu równomiernie. Minimalna wartość ε chroni przekształcenia spłaszczające, np. łodygę paproci.

Dopasowanie do okna. Najpierw liczymy kilka tysięcy punktów, aby wyznaczyć prostokąt otaczający atraktor. Dodajemy margines 5% i dobieramy skalę z zachowaniem proporcji, tak aby fraktal wypełniał okno.

Kolorowanie. Każde przekształcenie ma własny kolor, a punkt przyjmuje kolor przekształcenia, które go wyznaczyło. Opcjonalnie zliczamy trafienia w piksel i ustalamy jasność logarytmicznie: jasność $= \ln(1 + h)/\ln(1 + h_{\max})$.

5.3 Animacja przejścia między fraktalami IFS

Niewielka zmiana współczynników przekształceń daje niewielką zmianę obrazu. Dlatego klatkę $k = 0, 1, \ldots, K-1$ przejścia od fraktala A do fraktala B tworzymy, zmieniając liniowo każdy współczynnik:

$$w_i^{(k)} = (1 - t)\,w_i^A + t\,w_i^B, \qquad t = \frac{k}{K-1} \in [0, 1].$$

Pierwsza klatka przedstawia więc fraktal A, a ostatnia fraktal B.

- **Różna liczba przekształceń.** Do zbioru z mniejszą liczbą przekształceń dokładamy kopie istniejących, dzieląc prawdopodobieństwo kopiowanego przekształcenia na pół. Dwie identyczne kopie dają ten sam atraktor, więc pierwsza klatka się nie zmienia.
- **Poprawność klatek pośrednich.** Jeśli przekształcenia A i B są zwężające, to ich kombinacja wypukła też jest zwężająca: $\|(1-s)M_A + sM_B\| \le (1-s)\|M_A\| + s\|M_B\| < 1$. Każda klatka jest więc poprawnym fraktalem.
- **Stabilny kadr.** Prostokąt otaczający liczymy dla każdej klatki i wygładzamy go średnią kroczącą, aby obraz nie „pływał”.
- **Eksport.** Każde wywołanie `tick()` liczy jedną klatkę, zapisuje ją do pliku `klatka_NNNN.png` (lub `.bmp`) i pokazuje w oknie. Między klatkami interfejs obsługuje zdarzenia, więc działa podgląd, pasek postępu i przycisk *Przerwij*. W poszerzeniu klatki liczy wątek roboczy, a przerwanie sygnalizuje flaga `std::atomic<bool>`.

5.4 L-systemy i grafika żółwia

L-system składa się z aksjomatu, reguł przepisywania i kąta. W każdej iteracji wszystkie symbole napisu zastępujemy jednocześnie według reguł. Długość napisu rośnie wykładniczo, więc ograniczamy ją, np. do 5 mln symboli, i ostrzegamy użytkownika.

Symbol	Polecenie dla żółwia
`F`	idź naprzód o krok i rysuj odcinek
`f`	idź naprzód o krok bez rysowania
`+`, `-`	obróć się w lewo lub w prawo o zadany kąt
`[`	zapamiętaj pozycję i kierunek (odłóż na stos)
`]`	wróć do ostatnio zapamiętanego stanu (zdejmij ze stosu)

Żółw przechodzi napis dwa razy. Za pierwszym razem tylko wyznacza prostokąt otaczający rysunek, aby dobrać skalę. Za drugim rysuje odcinki algorytmem Bresenhama bezpośrednio w buforze pikseli.

5.5 Fraktale IFS 3D (poszerzenie)

Przekształcenie przestrzenne ma 12 współczynników: $\mathbf{x}' = M\mathbf{x} + \mathbf{t}$, gdzie M to macierz 3×3. Grę w chaos prowadzimy w przestrzeni. Każdy punkt przekształcamy do układu kamery i rzutujemy perspektywicznie: $x_s = f\,x_c / z_c + W/2$. Własny bufor głębi przechowuje dla piksela odległość najbliższego punktu, więc dalsze punkty nie przesłaniają bliższych. Jasność maleje z odległością od obserwatora.

5.6 Obliczenia wielowątkowe (poszerzenie)

Obraz dzielimy na poziome pasy, po jednym na wątek (`std::thread::hardware_concurrency()`). Każdy wątek zapisuje tylko swoje wiersze, więc nie potrzeba blokad. W grze w chaos każdy wątek ma własny generator liczb losowych i własny bufor trafień, które na końcu sumujemy. Obraz liczony na wielu wątkach musi być identyczny z obrazem liczonym na jednym, co sprawdzamy w testach.

6 Podział pracy i analiza czasowa

Dokumentację i testy prowadzimy wspólnie. Na początku wspólnie ustalamy część wspólną (`Image`, `LineReader`, `ParseError`, `FractalRenderer`) i szkielet aplikacji z zaślepkami trzech trybów. Od tej

chwili każdy pracuje w swoim module i widzi wynik w programie. Tygodnie liczymy od akceptacji tematu.

Etap	Jakub Piela interfejs, zbiory zespolone	Marcel Gładysz IFS i animacja	Patryk Pilch L-systemy
Tydz. 1–2 szkielet	szkielet aplikacji (CMake, okno, panel, menu, przełączanie trybów), nawigacja myszą, odwzorowanie piksel–płaszczyzna, Mandelbrot	parser `.ifs`, gra w chaos	parser `.lsys`, przepisywanie napisów
Tydz. 3–4 rdzeń	Julia (wybór c kliknięciem), płynne kolorowanie, palety	wbudowane fraktale (Sierpiński, paproć), edycja współczynników, autoskalowanie, kolorowanie	grafika żółwia z rozgałęzieniami, wbudowane krzywe, dopasowanie do okna
Tydz. 5–6 komplet podstaw	zapis obrazu PNG/BMP, okna dialogowe, integracja modułów	animacja przejścia dwóch fraktali, podgląd klatek, eksport ponumerowanych klatek	wczytywanie i zapis definicji, testy parserów
Tydz. 7–8 poszerzenia	wielowątkowość i animacja w tle z paskiem postępu, animacja przybliżania, zapis w wysokiej rozdzielczości, archiwum 7z	plik scenariusza, fraktale IFS 3D	kolejne zbiory zespolone (Burning Ship, Multibrot, Newton), edytor palet

Wymagania podstawowe mają być gotowe po szóstym tygodniu. Ostatnie dwa tygodnie zostają na poszerzenia, testy i dokumentację w PDF.

7 Plan testów

Testy modułów działają bez okna i uruchamiamy je poleceniem `ctest`.

- **Testy modułów:** parsery na poprawnych i celowo błędnych plikach (oczekiwany komunikat z numerem wiersza); znane wyniki: punkt $c = 0$ należy do zbioru Mandelbrota, a $c = 1$ ucieka po 3 krokach; liczba symboli `F` w krzywej Kocha po n iteracjach wynosi 5^n; współczynniki pierwszej i ostatniej klatki przejścia są równe współczynnikom fraktali A i B.
- **Testy powiązań:** zmiana trybu w trakcie generowania animacji, zapis obrazu po wczytaniu definicji, przerwanie eksportu animacji.
- **Testy całościowe:** przejście trójkąta Sierpińskiego w paproć Barnsleya w 100 klatkach, zmiana rozmiaru okna, pomiar czasu obliczeń klatki.
- **Niezmienniki:** liczba zapisanych klatek równa zadanej liczbie klatek przejścia; w poszerzeniu obraz zbioru zespolonego nie zależy od liczby wątków.

8 Ryzyka i plan awaryjny

- **Wydajność przy dużym przybliżeniu** – w wersji podstawowej liczymy w jednym wątku, więc podczas przeciągania pokazujemy podgląd w niższej rozdzielczości, a pełny obraz po puszczeniu przycisku myszy. Obliczenia wielowątkowe są pierwszym poszerzeniem.
- **Precyzja `double`** – przybliżanie ograniczamy do zakresu, w którym `double` wystarcza, i nie pozwalamy przybliżać dalej.
- **Różne systemy w zespole** – budowanie przez CMake, biblioteki jako submoduły w ustalonych wersjach, instrukcja budowania w `README.md`.
- **Bardzo długie napisy L-systemów** – limit długości i ostrzeżenie zamiast zawieszenia programu.
- **Zakres** – jeśli zabraknie czasu, wymagania podstawowe pozostają nietknięte, a odpadają pozycje z listy poszerzeń, od końca.